

The background features a dark blue gradient with a stylized, abstract illustration of a coffee cup. The cup is depicted with dark blue, swirling lines, and steam is shown as two thick, brown, wavy lines rising from the top. The overall aesthetic is modern and minimalist.

JaCoCo

Guida Completa alla Code Coverage in Java

Perché la Code Coverage?



Riduzione Bug

Identifica le aree di codice non testate dove potrebbero nascondersi difetti logici.



Fiducia nel Refactoring

Una copertura alta garantisce che le modifiche al codice non rompano funzionalità esistenti.



Conformità

Spesso richiesta come requisito di qualità in ambienti enterprise e regolamentati.

Cos'è JaCoCo?

JaCoCo (Java Code Coverage) è lo standard de facto nel mondo Java.

È una libreria leggera che si integra nella JVM per monitorare quali istruzioni vengono eseguite durante i test automatici. Supporta pienamente Java 8 fino alle versioni più recenti.



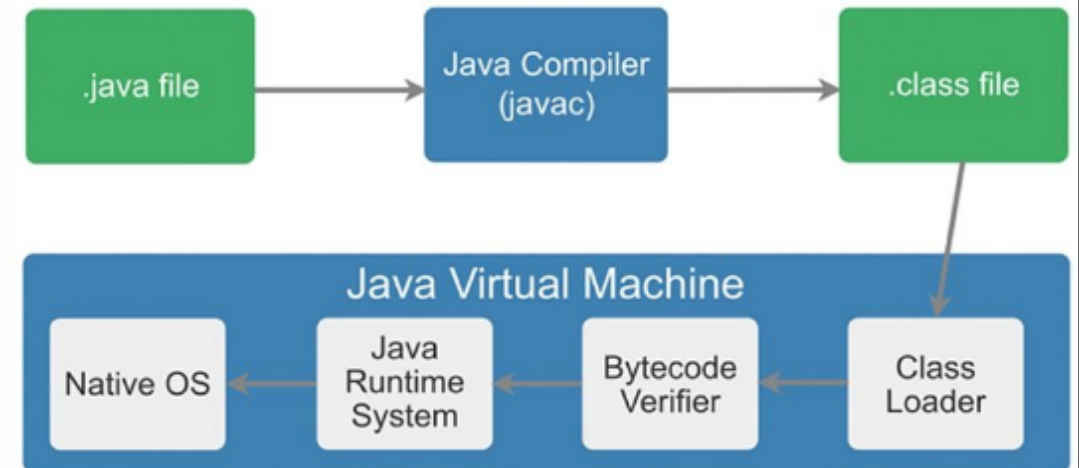
Architettura: Java Agent

Strumentazione On-the-fly

JaCoCo utilizza un **Java Agent** che si aggancia alla JVM all'avvio.

Modifica il **bytecode** in memoria mentre viene caricato dal ClassLoader, inserendo delle "sonde" (probes) che registrano l'esecuzione. Non richiede ricompilazione del codice sorgente.

What is Java bytecode?



Source: <http://www.techlila.com/write-programs-linux/>

Configurazione: Maven

pom.xml

```
<plugin>
  <groupId>org.jacoco<groupId>
  <artifactId>jacoco-maven-plugin<artifactId>
  <version>0.8.11<version>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent<goal>
        <goal>report<goal>
      <goals>
    <execution>
  <executions>
</plugin>
```

Configurazione: Gradle

build.gradle

```
plugins {  
    id 'java'  
    id 'jacoco'  
}  
  
test {  
    finalizedBy jacocoTestReport  
}  
  
jacocoTestReport {  
    dependsOn test  
}
```



Gradle rende l'integrazione ancora più semplice grazie al plugin nativo.

Analisi del Codice: Esempio

Analizziamo un metodo semplice che valuta un punteggio.

Il nostro test case passa il valore **10**.

Osserviamo come JaCoCo traccia l'esecuzione rispetto alla condizione limite di **12**.

ScoreValidator.java

```
public String validate(int points) {  
    if (points > 12) { // False: 10 non è > 12  
        return "Eccellente";  
    }  
    } else {  
        return "Standard";  
    }
```

Il Codice Colori



Verde (Full)

L'istruzione o il ramo è stato eseguito completamente durante il test.



Giallo (Partial)

L'istruzione è stata eseguita, ma solo una parte dei rami decisionali è stata percorsa.



Rosso (Missed)

L'istruzione non è mai stata raggiunta da nessun test.

Metrica 1: Line Coverage



La base della misurazione

Rappresenta il rapporto tra le righe di codice eseguite e il totale delle righe eseguibili.

$$\text{Coverage} = (\text{Righe Eseguite} / \text{Totale Righe}) * 100$$

È la metrica più semplice ma anche la più ingannevole: eseguire una riga non significa averne verificato la correttezza logica.

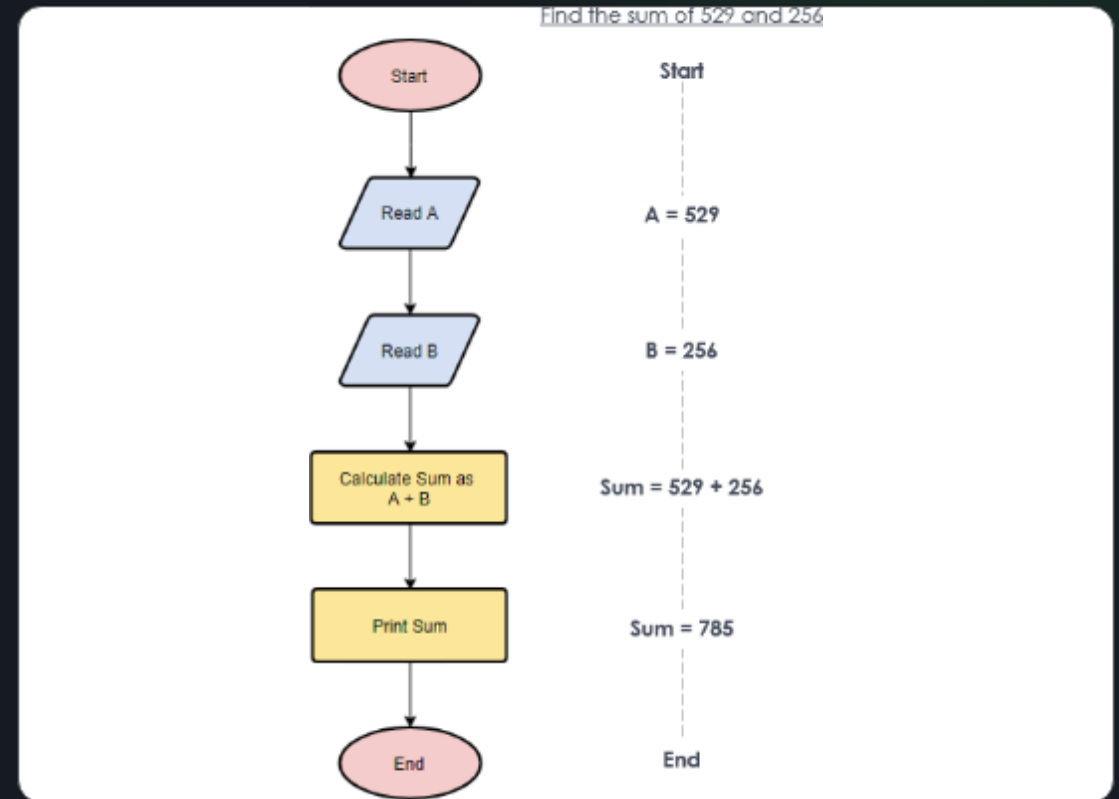
Metrica 2: Branch Coverage

Decisioni e Flussi

Molto più importante della Line Coverage.

Verifica se sono stati percorsi tutti i possibili esiti di una struttura di controllo (if, switch).

Un if ($x > 0$) ha due rami: *True* e *False*. Entrambi devono essere testati per avere il 100%.



Metrica 3: Complessità Ciclomatica



La complessità del codice

Misura il numero di percorsi indipendenti attraverso il codice sorgente.

JaCoCo calcola la complessità per ogni metodo. Se un metodo ha una complessità alta (es. > 10) e una copertura bassa, è un forte candidato per il refactoring e bug nascosti.

Formula: $V(G) = E - N + 2$

Il Report HTML

Il risultato finale è un sito statico interattivo.

Permette di navigare:

- ✓ Riepilogo globale
- ✓ Dettaglio per Package
- ✓ Dettaglio per Classe
- ✓ Sorgente evidenziato

Element	Missed Instructions	Cov.	Missed Branches	Cov.
com.parkit.parkingsystem.service		73 %		57 %
com.parkit.parkingsystem.config		8 %		0 %
com.parkit.parkingsystem.dao		81 %		62 %
com.parkit.parkingsystem.util		19 %		0 %
com.parkit.parkingsystem.model		70 %		20 %
com.parkit.parkingsystem		0 %		n/a
com.parkit.parkingsystem.constants		80 %		n/a
Total	366 of 1 128	67 %	35 of 61	42 %

Element	Missed Instructions	Cov.	Missed Branches	Cov.
com.parkit.parkingsystem.dao		2 %		0 %
com.parkit.parkingsystem.service		43 %		36 %
com.parkit.parkingsystem.config		0 %		0 %
com.parkit.parkingsystem.model		59 %		10 %
com.parkit.parkingsystem.util		19 %		0 %
com.parkit.parkingsystem		0 %		n/a
com.parkit.parkingsystem.constants		80 %		n/a
Total	813 of 1 128	27 %	48 of 61	21 %

Filtrare il Rumore



Cosa escludere?

Non ha senso testare codice senza logica o generato automaticamente:

- ✓ DTO / POJO (Getter/Setter)
- ✓ Classi di Configurazione
- ✓ Codice generato (Lombok, MapStruct)

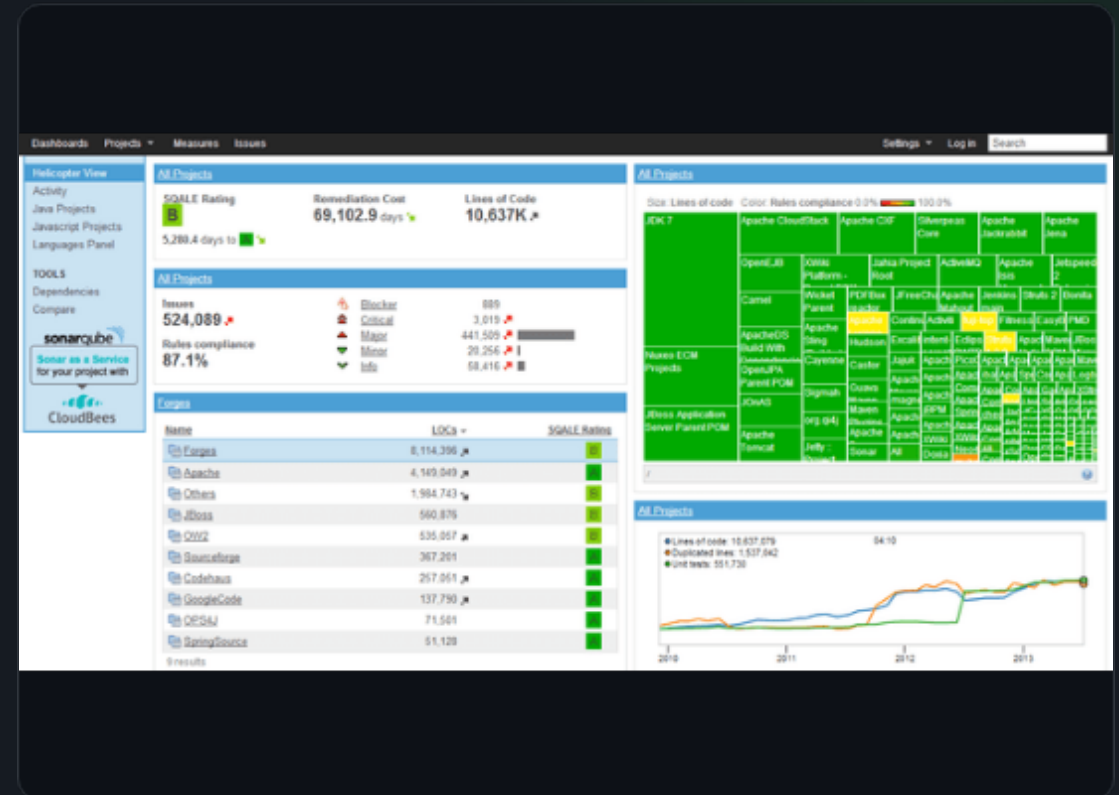
pom.xml (Exclusions)

```
<configuration>
  <excludes>
    <exclude>**/dto/*exclude>
    <exclude>**/model/*exclude>
    <exclude>**/config/*exclude>
  </excludes>
</configuration>
```

Dashboard CI/CD

JaCoCo non vive in isolamento. I suoi dati (XML) alimentano dashboard centralizzate come **SonarQube**.

Questo offre una visione storica dell'andamento della qualità del progetto, permettendo di vedere se la copertura sta migliorando o peggiorando nel tempo.



Quality Gates



"You Shall Not Pass"

Imposta regole rigide nella tua pipeline di build.

COVERAGE < 80% => BUILD FAILED

NEW CODE COVERAGE < 90% => BUILD FAILED

Blocca il merge di Pull Request che abbassano la qualità generale del codice.

Best Practices



Focus sulla Business Logic

Prioritizza i test sui "Service Layer" e algoritmi complessi.



Testare i Rami

Scrivi test case specifici per entrare negli else e nei casi limite (edge cases).



Automazione

Non eseguire JaCoCo manualmente. Deve essere parte integrante di ogni build Maven/Gradle.

| Trappole Comuni

- ✓ **Ossessione del 100%:** Raggiungere il 100% spesso comporta test inutili e fragili. L'80% è un ottimo standard.
- ✓ **Test senza Asserzioni:** È possibile avere 100% di copertura chiamando i metodi senza verificare il risultato (assert). Questo è inutile.
- ✓ **Ignorare le eccezioni:** Molti test coprono il "happy path" ma dimenticano i flussi di errore (catch blocks).

